

Binary Search Revision Notes

Striver SDE Sheet - Short approach notes for quick interview revision

Core Binary Search Mindset

- Binary search is used on sorted indexes or on a possible answer range.
- For every problem, define low, high, mid, and what condition(mid) means.
- When condition is possible, decide whether to minimize or maximize the answer.

1. Nth Root of an Integer

Pattern	Binary search on answer
Approach	Search x from 1 to m. Compute mid^n . If equal, return mid; if smaller, move right; if greater, move left.
Intuition	The answer is a value, not an index. We test whether a number can be the nth root.
Complexity	$O(\log m * n)$ time, $O(1)$ space
Remember	Use long while multiplying to avoid overflow.

2. Matrix Median

Pattern	Binary search on value
Approach	Search between minimum and maximum matrix value. For each mid, count elements $\leq$ mid using upper bound in every row.
Intuition	If count is less than or equal to required median position, median is larger; otherwise it is smaller.
Complexity	$O(\log(\text{max-min}) * \text{rows} * \log(\text{cols}))$ time, $O(1)$ space
Remember	Rows are sorted, so upper bound works row-wise.

3. Single Element in Sorted Array

Pattern	Binary search on pair pattern
Approach	Before the single element, pairs start at even index. After it, the pattern breaks. Use this to move left or right.
Intuition	The single element disturbs the even-odd pair structure.
Complexity	$O(\log n)$ time, $O(1)$ space
Remember	Trick: compare $\text{arr}[\text{mid}]$ with $\text{arr}[\text{mid} \oplus 1]$.

4. Search in Rotated Sorted Array

Pattern	Modified binary search
Approach	At every mid, identify which half is sorted. If target lies in sorted half, search there; otherwise search the other half.
Intuition	Even after rotation, one half always remains sorted.
Complexity	$O(\log n)$ time, $O(1)$ space
Remember	First find sorted half, then check target range.

5. Median of Two Sorted Arrays

Pattern	Binary search on partition
Approach	Binary search on smaller array. Partition both arrays so left side has half elements and all left values $\leq$ all right values.
Intuition	A correct partition directly gives the median without merging.
Complexity	$O(\log(\min(n, m)))$ time, $O(1)$ space
Remember	Use <code>INT_MIN</code> and <code>INT_MAX</code> for boundary partitions.

6. Kth Element of Two Sorted Arrays

Pattern	Binary search on partition
Approach	Place exactly k elements on the left side by cutting both arrays. Valid partition gives $\max(\text{left1}, \text{left2})$.
Intuition	Same idea as median, but the left side size is k.
Complexity	$O(\log(\min(n,m)))$ time, $O(1)$ space
Remember	Bounds: $\text{low} = \max(0, k-m)$, $\text{high} = \min(k, n)$.

7. Allocate Minimum Number of Pages

Pattern	Binary search on answer - minimize maximum
Approach	Search answer from $\max(\text{pages})$ to $\text{sum}(\text{pages})$. Check how many students are needed if max pages allowed is mid.
Intuition	If allocation is possible with $\leq k$ students, try smaller maximum pages.
Complexity	$O(n \log(\text{sum}))$ time, $O(1)$ space
Remember	This is the classic minimize-the-maximum pattern.

8. Aggressive Cows

Pattern	Binary search on answer - maximize minimum
Approach	Sort stalls. Search distance. Greedily place cows with at least mid distance between them.
Intuition	If cows can be placed with distance mid, try a larger minimum distance.
Complexity	$O(n \log(\text{maxDistance}))$ time, $O(1)$ space
Remember	This is the classic maximize-the-minimum pattern.

Binary Search Pattern Summary

Problem Type	Pattern / Trick
Nth Root	Search possible answer
Matrix Median	Search value range + count $\leq$ mid
Single Element	Even-odd pair pattern
Rotated Array Search	Find sorted half
Median / Kth of Two Arrays	Partition-based binary search
Allocate Pages	Minimize maximum
Aggressive Cows	Maximize minimum

Final Checklist

- Is the problem asking for exact search or best possible answer?
- Can I write a $\text{condition}(\text{mid})$ function?
- If condition is true, should I move left for minimum or right for maximum?